

Skillab

AI Agent Development

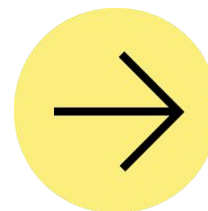
Richard Șerban Francu

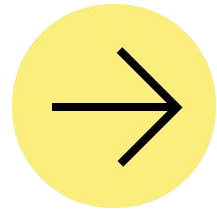
Senior AI Engineer @UCE Systems



Programa cursului

1. Noțiuni de bază — API-uri LLM și primul agent
2. Instrumente, prompturi și observabilitate
3. Procesarea și extragerea documentelor
4. Stocarea bazelor de date și RAG
5. LangGraph — Stare și fluxuri de lucru
6. Orchestrarea multi-agent
7. Optimizarea ML — Dincolo de apelurile LLM
8. Memorie și cache
9. Securitate și măsuri de protecție
10. Integrarea MCP
11. Inferența SageMaker
- 12. Training, Fine-Tuning și RLHF**





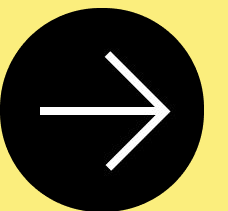
Training, Fine-Tuning şı RLHF

Lecția 12

Training, Fine-Tuning și RLHF

- Înțelegerea RLHF și a modului în care acesta modelează comportamentul LLM
- Implementarea fine-tuning-ului cu instrumente moderne
- Configurarea sarcinilor de training pe AWS

Fine-tuning vs Prompt Engineering



Fine-tuning sau Prompt Engineering?

Întrebarea corectă nu e „care e mai puternic”, ci „ce problemă am de fapt”.

ÎNCEPI AICI

Prompt Engineering

Rezolvă ~80% din cazuri la o fracțiune din cost.

Bun pentru: instrucțiuni clare, few-shot, format de output, lanțuri de raționament.

Iterezi în secunde, nu în ore. Zero infrastructură, zero date de antrenare.

CÂND AI UN MOTIV

Fine-tuning

Schimbi comportamentul modelului, nu doar instrucțiunea.

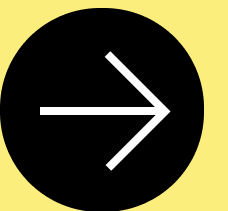
Merită când: stil/format consistent la scară, domeniu îngust, latență & cost de redus, comportament greu de prins în prompt.

Cost: date etichetate, compute, un ciclu de antrenare.

⚠ **Capcana clasică:** fine-tuning **NU** adaugă cunoștințe factuale noi în mod fiabil — pentru „modelul să știe documentele mele” folosești **RAG**, nu fine-tuning. Fine-tuning modelează **cum** răspunde, nu **ce** știe.

Default = prompt engineering. Treci la fine-tuning doar când ai un motiv structural pe care promptul nu-l poate rezolva.

Tehnici Fine-tuning



Full Fine-tuning → LoRA → QLoRA

Aceeași idee, costuri diferite: cât din model atingi de fapt când antrenezi?

FULL FT

Full Fine-tuning

Antrenezi **toți parametrii** modelului.

100% din weights actualizate. Calitate maximă, dar memorie uriașă: model + gradienti + optimizer states.

7B ≈ 80GB+ VRAM. Rar justificat.

LORA

LoRA

Îngheți modelul, antrenezi **matrice mici adăugate**.

Doar ~0.1–1% din parametri devin trainable. Weights originale rămân neatinse.

Un adapter de câțiva MB în loc de model întreg.

QLORA

QLoRA

LoRA peste un model **cuantizat pe 4-bit**.

Modelul de bază comprimat în memorie, adapterele rămân în precizie plină.

8B încapă pe un singur GPU 24GB. Sweet spot practic.

De reținut: toate trei produc același tip de rezultat — un model adaptat. Diferă doar **câtă memorie** îți cer. În practică pornești cu **LoRA / QLoRA** și treci la full FT doar dacă chiar ai nevoie.

Parameter-efficient = antrenezi 1% și obții ~95% din câștig. Asta face fine-tuning accesibil pe un singur GPU.

SFT vs DPO: două obiective diferite

SFT învață modelul SĂ răspundă. DPO îl învață CARE răspuns e preferat.

SFT — Supervised Fine-Tuning

Imitație. Îi dai perechi (**prompt** → **răspuns bun**) și modelul învață să le reproducă. E baza oricărui fine-tuning — exact ce face demo-ul nostru.

```
1 from trl import SFTTrainer
2
3 trainer = SFTTrainer(
4     model=model,
5     train_dataset=ds, # (prompt → răspuns)
6     peft_config=lora,
7 )
8 trainer.train()
9
```

DPO — Direct Preference Optimization

Preferințe. Îi dai triplete (**prompt** → **ales / respins**) și modelul învață să prefere unul. Aliniere fără reward model separat — mai simplu decât RLHF.

```
1 from trl import DPOTrainer
2
3 trainer = DPOTrainer(
4     model=model,
5     train_dataset=ds, # (chosen / rejected)
6     peft_config=lora,
7 )
8 trainer.train()
```

Pipeline tipic: SFT întâi, apoi DPO pentru aliniere. Același API trl, schimbi doar trainerul și forma datelor.

Ecosistemul de unelte

Patru librării, roluri diferite — nu sunt concurente, lucrează împreună.

TRL

trl

Librăria Hugging Face pentru fine-tuning. Îți dă SFTTrainer, DPOTrainer, PPOTrainer — interfața standard prin care antrenezi. Stratul pe care îl atingi direct.

PEFT

PEFT / LoRA

Implementarea parameter-efficient fine-tuning. LoraConfig de aici se plimbă în SFTTrainer printr-un singur argument. Adapterele mici vin din PEFT.

UNSLOTH

Unsloth

Backend optimizat: aceeași antrenare, mai puțină memorie, pași mai rapizi. Drop-in — în demo e doar un flag `--use_unsloth`. Bun pentru GPU mic.

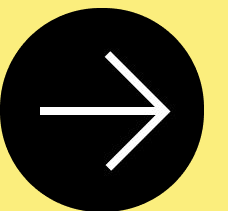
AXOLOTL

Axolotl

Wrapper config-first: definești totul într-un YAML, fără să scrii cod de training. Practic pentru experimente repetabile și rețete partajate în echipă.

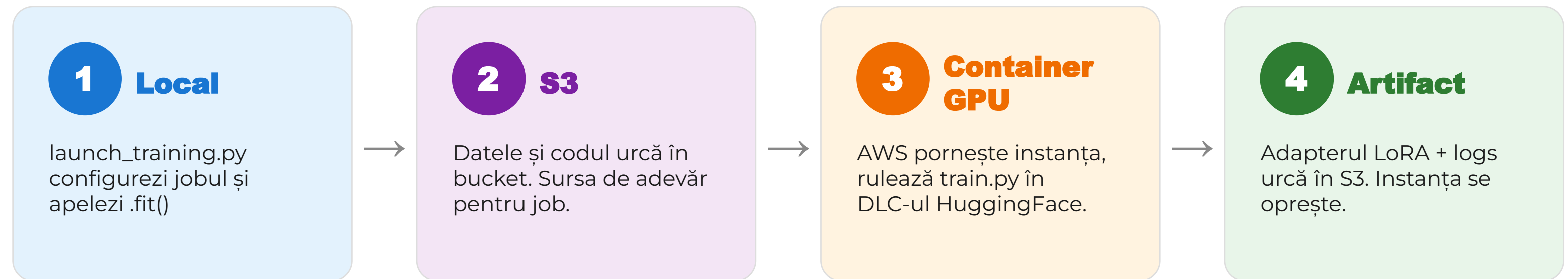
În demo-ul nostru: trl + PEFT fac munca, Unsloth e un flag opțional. Asta acoperă 90% din cazuri.

Fine-tuning în SageMaker



Cum rulează un training job pe SageMaker

Tu lansezi de pe laptop. AWS pornește un GPU, antrenează, salvează — și-l stinge.



De ce contează modelul ăsta

Nu plătești un GPU care stă aprins. Instanța trăiește doar cât durează jobul — pornește, antrenează, salvează în S3, se stinge automat. Tu rămâi cu un adapter de câțiva MB și plătești doar minutele de training. Codul de pe laptop e doar **o rețetă de lansare**; munca grea se întâmplă pe instanța AWS.

Managed training = lansezi un job, nu administrezi un server. Asta e toată ideea SageMaker.

HuggingFace Estimator: lansarea jobului

launch_training.py — rulează pe laptopul tău și descrie jobul pe care-l vrei.

```
1 estimator = HuggingFace(  
2     entry_point="train.py",           # ce rulează în container  
3     source_dir="src",                 # din ce folder  
4     role=ROLE,  
5     instance_type="ml.g5.2xlarge",    # 1xA10G 24GB – potrivit pentru ~1B  
6     instance_count=1,  
7     transformers_version="4.49.0",    # alege imaginea DLC pre-construită  
8     pytorch_version="2.5.1",  
9     py_version="py311",  
10    hyperparameters=hyperparameters,  
11    metric_definitions=METRIC_DEFINITIONS,  
12    tensorboard_output_config=tb_config,  
13    use_spot_instances=False,          # True + max_wait => -60-70% cost  
14    max_run=3 * 3600,  
15 )  
16  
17 # .fit() împachetează tot și trimite jobul la AWS  
18 estimator.fit({"train": TRAIN_S3}, job_name=job_name, wait=True)  
19  
20
```

entry_point + source_dir

Ce script rulează în container și din ce folder. train.py e codul tău.

instance_type

Ce GPU închiriezi. ml.g5.2xlarge = 1xA10G 24GB, potrivit pentru ~1B.

transformers / pytorch_version

Aleg imaginea DLC pre-construită. Nu instalezi tu PyTorch + CUDA.

metric_definitions

Regex care extrage loss/lr din log și le trimite în CloudWatch.

tensorboard_output_config

Sincronizează curbele de loss în S3, live, în timpul jobului.

Estimatorul e un formular: ce cod, ce GPU, ce versiuni, ce să monitorizezi. .fit() trimite totul la AWS.

train.py: ce rulează în container

Aici trăiește fine-tuning-ul real: LoRA peste model + SFTTrainer din trl.

```
1  # CE antrenezi: adaptere mici peste proiecțiile de attention
2  peft_config = LoraConfig(
3      r=16,                # rangul LoRA – capacitatea adapterelor
4      lora_alpha=32,        # scalarea contribuției (regula: alpha ≈ 2×r)
5      lora_dropout=0.05,
6      bias="none",
7      task_type="CAUSAL_LM",
8      target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
9                      "gate_proj", "up_proj", "down_proj"],
10 )
11
12 # CUM antrenezi
13 sft_config = SFTConfig(
14     output_dir=OUTPUT_DIR,
15     num_train_epochs=EPOCHS,
16     per_device_train_batch_size=2,
17     gradient_accumulation_steps=8,    # batch efectiv = 2 × 8 = 16
18     learning_rate=2e-4,
19     lr_scheduler_type="cosine",
20     bf16=True,
21     gradient_checkpointing=True,
22     packing=True,                # umple secvențele la max_seq_len
23     max_seq_length=1024,
24     dataset_text_field="text",
25     logging_steps=5,
26     report_to="tensorboard",
27 )
```

r = 16

Rangul LoRA: cât de mari sunt matricele adăugate. Mai mare = mai multă capacitate.

lora_alpha = 32

Scalează contribuția adapterelor. Regulă uzuală: $\alpha \approx 2 \times r$.

target_modules

Ce straturi primesc adaptere — proiecțiile de attention (q, k, v, o).

batch × grad_accum

2×8 = batch efectiv 16, fără să ceri memorie pentru 16 deodată.

packing = True

Umple secvențele la max_seq_len — mai puține tokenuri irosite, training mai eficient.

LoraConfig spune CE antrenezi, SFTConfig spune CUM. SFTTrainer le pune cap la cap.

Instanța potrivită & monitorizare live

Alegi instanța după mărimea modelului, apoi urmărești jobul în timp real.

ALEGI INSTANȚA DUPĂ MODEL

~1B

ml.g5.2xlarge

1×A10G 24GB
LoRA simplu, fără cuantizare. Punctul de plecare din demo.

~8B

ml.g5.4xlarge

1×A10G 24GB
Adaugi `--use_4bit` (QLoRA) ca să încapă în 24GB.

+ `--spot` → 60–70% reducere de cost pe spot instances.

URMĂREȘTI JOBUL LIVE

TensorBoard

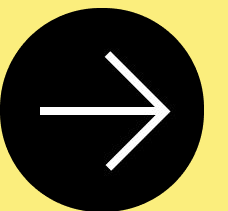
Curbele de loss, learning rate, grad_norm — streamuite din S3 cât rulează jobul. Zero setup: e cablat prin `TensorBoardOutputConfig`.

CloudWatch

Aceleași scalare ca metrice numite + metrice de sistem (GPU, memorie) — dovada că instanța chiar muncește. Cablat prin `metric_definitions`.

Ambele monitoare sunt zero-setup și stau în AWS. TensorBoard pentru loss, CloudWatch pentru GPU.

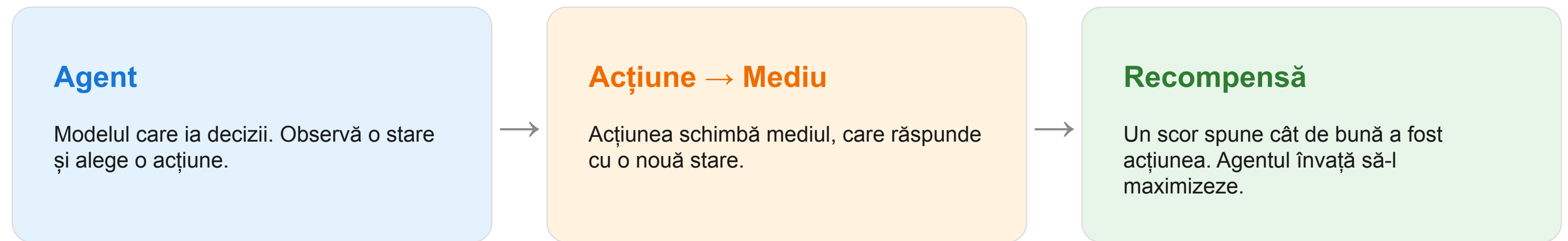
Reinforcement Learning



Reinforcement Learning pe scurt

Înainte de RLHF: ce e RL în sine? Înveți prin încercare și recompensă, nu din exemple etichetate.

BUCLA DE BAZĂ



La LLM-uri (RLHF), bucla e „colapsată”: modelul dă un singur răspuns și primește un scor — nu există un mediu care evoluează peste mai multe runde. Mecanica RL rămâne, bucla pas-cu-pas nu.

Supervised (SFT)

Profesorul îți dă răspunsul corect pentru fiecare exemplu. Înveți copiind. Simplu, dar ai nevoie de etichete pentru tot.

Reinforcement (RL)

Nimeni nu-ți spune răspunsul perfect — doar cât de bine te-ai descurcat. Înveți încercând. Potrivit când „corect” e greu de definit, dar „mai bun” e ușor de judecat.

Pentru LLM-uri, „recompensa” vine din preferințe umane. Exact asta face RLHF — slide-ul următor.

RLHF: înveți din feedback, nu din exemple

Reinforcement Learning from Human Feedback — RL aplicat la LLM-uri. Așa a fost aliniat ChatGPT.

Limita lui SFT

SFT imită exemple „bune”. Dar pentru multe întrebări nu există **un singur** răspuns corect — vrei „mai degrabă ăsta decât ăla”.

Ideea RL

Definești o **recompensă** pentru răspunsuri bune și lași modelul să-și ajusteze comportamentul ca să o maximizeze. Înveți din scor, nu din copiere.

PIPELINE-UL RLHF CLASIC

1 SFT

Pornești de la un model deja fine-tunat supervizat. Baza pe care construiești — exact ce am făcut în demo.

2 Reward Model

Antrenezi un al doilea model care învață din comparații umane (ales vs respins) să dea un scor oricărui răspuns.

3 PPO

Algoritmul RL ajustează modelul ca să crească scorul dat de reward model, fără să se îndepărteze prea mult de bază.

DPO = scurtătura modernă: aceeași aliniere din preferințe, fără reward model + PPO separate. De aia e popular.

Recap — Lecția 12: Fine-tuning & RL

DECIZIE

Fine-tune vs prompt — *promptul rezolvă ~80%; fine-tuning doar cu motiv structural (nu pentru cunoștințe → RAG)*

TEHNICI

LoRA / QLoRA — *antrenezi ~1% din parametri; QLoRA bagă 8B pe un GPU 24GB*

OBIECTIV

SFT vs DPO — *SFT imită exemple; DPO învață din preferințe (ales/respins)*

PIPELINE

SageMaker — *HuggingFace Estimator lansează jobul; train.py rulează LoRA+SFTTrainer în container*

OPS

Monitorizare — *TensorBoard pentru loss, CloudWatch pentru GPU — ambele zero-setup*

OVERVIEW

RLHF — *SFT → Reward Model → PPO; alinierea din feedback uman, scurtată azi de DPO*

Default prompt → fine-tune cu LoRA când ai motiv → rulezi pe SageMaker → monitorizezi live. Asta e tot fluxul.